Hash-Based Digital Signatures: From Lamport OTS to Merkle Signature Schemes

Applied Cryptography (CS 232) — Project Report

Aicha Slaitane
aicha.slaitane@kaust.edu.sa

Abdullah Algethami
abdullah.algethami@kaust.edu.sa

Abstract

We design, implement, and evaluate a complete hash-based digital signature system in Python. Starting from the foundational Lamport One-Time Signature (OTS) scheme, we progress through Winternitz OTS (WOTS) and culminate in a full Merkle Signature Scheme (MSS) that lifts the one-time restriction. We conduct systematic benchmarks across a wide parameter space—varying the Winternitz parameter $w \in \{4, \ldots, 256\}$ and tree heights $h \in \{1, \ldots, 8\}$—to quantify the signature-size versus computation-time tradeoff. We demonstrate the critical key-reuse vulnerability of Lamport signatures through a quantitative attack simulation, recovering the full 512-block secret key after only 20 reuses and validating the theoretical forgery probability $(1 - 2^{-k})^{256}$ against empirical data across 200 independent trials per configuration. Finally, we extend the construction to a two-layer *hypertree*, the same multi-layer Merkle aggregation idea used internally by SPHINCS+/SLH-DSA, and show that it reduces initial KeyGen time by an order of magnitude at matched signing capacity (e.g., 8× faster at $N = 256$ and 16× faster at $N = 1024$).

1 Introduction

Digital signatures are a cornerstone of modern information security, providing authentication, integrity, and non-repudiation. Classical schemes such as RSA and ECDSA derive their security from the computational hardness of integer factorization and the discrete logarithm problem, respectively. However, Shor's algorithm [1] demonstrates that a sufficiently powerful quantum computer can solve both problems in polynomial time, rendering these schemes insecure in a post-quantum world.

Hash-based signatures offer a fundamentally different security foundation. Their security rests on a single, well-studied assumption: the *one-wayness* and *(second-)preimage resistance* of the underlying cryptographic hash function. The best known quantum speedup against generic hash functions is Grover's algorithm, which provides only a quadratic advantage—reducing an n-bit hash from 2^n to $2^{n/2}$ security. For SHA-256, this yields approximately 128 bits of quantum security, aligning with NIST's Category-1 post-quantum requirement. This robustness is precisely why NIST standardized the hash-based scheme SLH-DSA (SPHINCS+) alongside the lattice-based ML-DSA in 2024.

In this project, we implement the fundamental building blocks that underpin modern hash-based signature standards: Lamport OTS, Winternitz OTS, and the Merkle Signature Scheme. All implementations use SHA-256 and are written in pure Python with no external cryptographic libraries. We accompany each construction with unit tests, systematic benchmarks, and a thorough security analysis of the key-reuse vulnerability inherent in OTS schemes.

2 Methods

2.1 Lamport One-Time Signatures

Proposed by Lamport in 1979 [2], the scheme generates a secret key consisting of n pairs of random blocks $\{(\mathrm{SK}[i][0],\ \mathrm{SK}[i][1])\}_{i=0}^{n-1}$, where n is the hash output length in bits (256 for SHA-256) and each block is 32 bytes. The public key is the element-wise hash: $\mathrm{PK}[i][b] = H(\mathrm{SK}[i][b])$. To sign a message m, the signer computes $h = H(m)$ and reveals $\mathrm{SK}[i][h_i]$ for each bit h_i. Verification rehashes each revealed block and compares against the public key. The resulting signature contains 256 blocks of 32 bytes each, totaling 8192 bytes, and the public key is twice that size (16384 bytes) since it stores both hash images per row.

2.2 Winternitz OTS (WOTS)

WOTS generalizes Lamport by processing the message hash in base-w digits rather than individual bits. The secret key consists of $\ell = \ell_1 + \ell_2$ random blocks, where $\ell_1 = \lceil 256/\log_2 w \rceil$ covers the message digits and $\ell_2 = \lfloor \log_2(\ell_1(w-1))/\log_2 w \rfloor + 1$ covers a checksum. The checksum prevents forgery by digit reduction: decreasing any message digit would require increasing the checksum digits, which requires additional hash iterations the attacker cannot reverse. The public key is obtained by hashing each secret block $w - 1$ times: $\mathrm{PK}[i] = H^{w-1}(\mathrm{SK}[i])$. To sign, each block is hashed d_i times; to verify, the verifier applies $w - 1 - d_i$ additional hashes and checks against PK. This reduces signature size from $\ell_1 + \ell_2$ blocks (vs. Lamport's $2n$), but increases computation since each chain requires up to $w - 1$ hash evaluations.

2.3 Merkle Signature Scheme

OTS schemes are inherently single-use. The Merkle tree construction [3] overcomes this by generating $N = 2^h$ independent OTS keypairs, hashing each OTS public key into a leaf node, and building a binary hash tree whose root serves as the long-term public key. A signature for the i-th message includes: (a) the OTS signature, (b) the OTS public key, and (c) an *authentication path*—the h sibling hashes along the path from leaf i to the root. The verifier checks the OTS signature and reconstructs the root from the leaf using the authentication path, accepting if and only if the reconstructed root matches the published public key. A `next_leaf` counter enforces strict one-time usage of each leaf, raising an error upon exhaustion.

3 Experimental Results

All benchmarks were collected on a local Linux workstation using SHA-256. Timing measurements are medians over 5 independent runs.

3.1 OTS Primitive Comparison

We benchmarked Lamport OTS and WOTS across all power-of-two Winternitz parameters from $w = 4$ to $w = 256$. Table 1 summarizes the key generation, signing, verification times, and key/signature sizes.

Scheme	KG (ms)	Sign (ms)	Vrfy (ms)	Sig (B)	PK (B)
Lamport	0.53	0.03	0.17	8192	16384
WOTS $w=4$	0.29	0.15	0.15	4256	4256
WOTS $w=8$	0.38	0.18	0.21	2880	2880
WOTS $w=16$	0.56	0.30	0.28	2144	2144
WOTS $w=32$	0.92	0.47	0.47	1760	1760
WOTS $w=64$	1.49	0.68	0.84	1440	1440
WOTS $w=128$	2.53	1.27	1.31	1248	1248
WOTS $w=256$	4.47	2.28	2.29	1088	1088

Table 1: OTS performance across Winternitz parameters. KG = KeyGen, Vrfy = Verify, Sig = signature size, PK = public key size.

The Winternitz trade-off is clearly visible in both Table 1 and Figures 1–2. As w increases, signature size decreases logarithmically (Figure 1): WOTS with $w = 256$ is 7.5× smaller than Lamport (1088 B vs. 8192 B). This reduction arises because each doubling of w groups more bits per digit, reducing the number of hash chains ℓ_1 by a factor proportional to $\log_2 w$. However, computation cost grows linearly with w (Figure 2), since each chain is $w - 1$ hashes long: WOTS with $w=256$ is approximately 75× slower to sign than Lamport. Note that Lamport's signing is extremely fast (0.03 ms) because it involves no hash computation at all—only selecting and copying precomputed blocks.

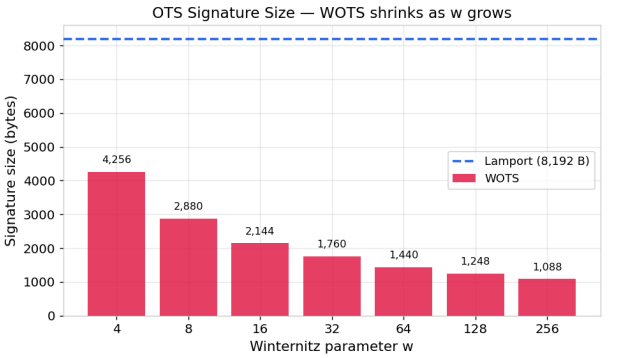

Figure 1: OTS signature size vs. w. The logarithmic decrease reflects that each doubling of w halves the number of base-w digits needed to represent the 256-bit message hash.

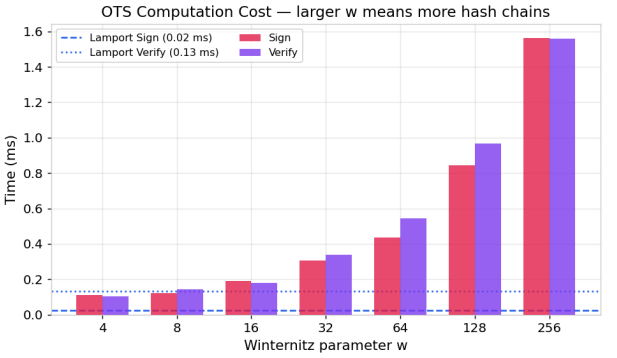

Figure 2: OTS sign and verify times vs. w. The linear growth confirms that computation is dominated by the chain length $w - 1$.

3.2 MSS Height Sweep

Table 2 reports MSS performance for tree heights $h = 1$ to 8 using both Lamport and WOTS ($w=16$) as the leaf OTS.

Config	N	KG (ms)	Sign (ms)	Vrfy (ms)	Sig (B)
$h=2$, Lamp.	4	2.49	0.04	0.27	24644
$h=4$, Lamp.	16	10.04	0.05	0.28	24708
$h=6$, Lamp.	64	41.65	0.05	0.27	24772
$h=8$, Lamp.	256	174.39	0.10	0.30	24836
$h=2$, WOTS	4	2.30	0.30	0.30	4356
$h=4$, WOTS	16	9.08	0.30	0.30	4420
$h=6$, WOTS	64	36.31	0.32	0.30	4484
$h=8$, WOTS	256	144.46	0.31	0.30	4548

Table 2: MSS performance vs. tree height ($N = 2^h$ leaves).

Three key observations emerge from Table 2 and Figures 3–6. First, as shown in Figure 3, KeyGen scales as $O(2^h)$: going from $h = 4$ to $h = 8$ increases the cost by roughly 16×, confirming that the dominant expense is materializing a full OTS keypair at

every leaf. Second, Sign and Verify times remain effectively constant with respect to h (Figures 4 and 5), because the additional h hash operations for the authentication path are negligible compared to the underlying OTS work. Third, signature size grows by only 32 bytes per tree level (Figure 6)—one SHA-256 sibling hash—meaning that doubling the signing capacity ($h \to h+1$) costs a trivial 32 B overhead per signature. Across all heights, WOTS leaves produce roughly 5.5× smaller MSS signatures than Lamport leaves (∼4.5 KB vs. ∼24.7 KB).

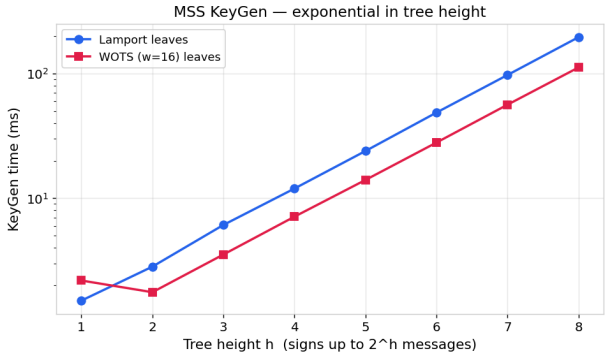

Figure 3: MSS KeyGen time vs. tree height on a log scale. The near-perfect linear trend on the log-scale confirms the $O(2^h)$ scaling.

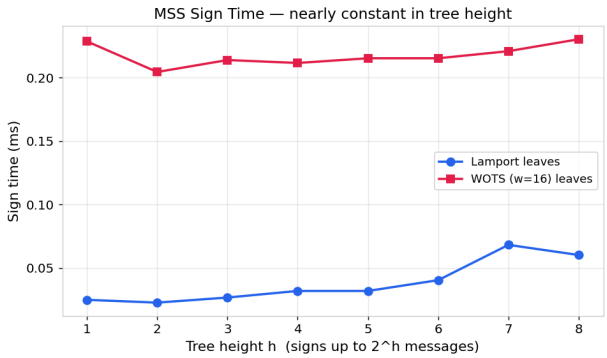

Figure 4: MSS Sign time vs. height. Signing remains flat regardless of tree size, as the h additional hashes are negligible next to the OTS operations.

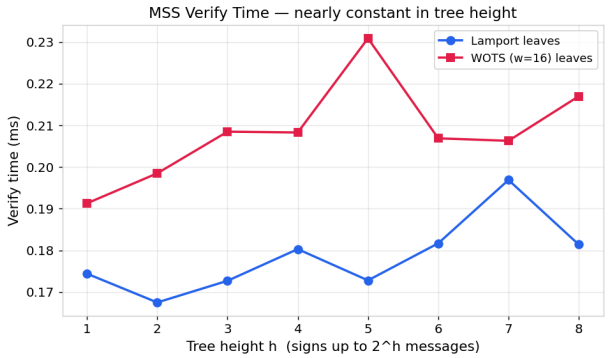

Figure 5: MSS Verify time vs. height. Like signing, verification cost is dominated by the OTS check and is independent of h.

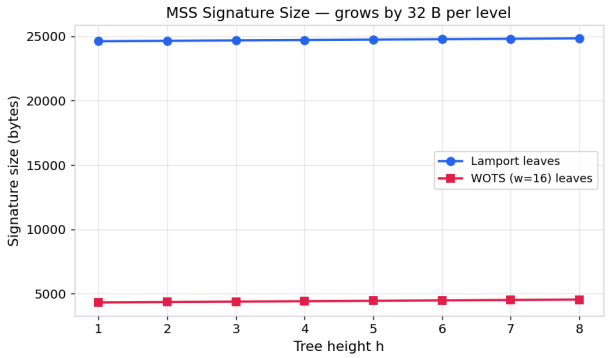

Figure 6: MSS signature size vs. height. Growth is strictly linear at 32 B per level, independent of the OTS backend choice.

3.3 Parameter Tuning: MSS with Varying w

Fixing $h=4$ (16 leaves), we varied the WOTS parameter to study the size-vs-speed trade-off within the full MSS context (Table 3). Going from $w=4$ to $w=256$ shrinks signatures by 3.7× (8644 B to 2308 B) but increases total computation (sign + verify) by 13×. Figure 7 visualizes this Pareto frontier, showing the smooth and predictable nature of the trade-off curve. The "knee" of the curve sits near $w=16$, which is why real-world standards (XMSS, SPHINCS+) adopt this value.

w	KG (ms)	Sign (ms)	Vrfy (ms)	Sig (B)
4	4.95	0.16	0.18	8644
16	9.09	0.30	0.30	4420
64	23.52	0.65	0.84	3012
256	70.59	2.26	2.26	2308

Table 3: MSS ($h=4$) performance vs. Winternitz parameter w.

Figure 7: MSS size-vs-speed trade-off at $h=4$. The knee near $w = 16$ marks the practical sweet spot adopted by XMSS and SPHINCS+.

Based on the complete sweep, Table 4 summarizes optimal parameter choices for four representative deployment priorities.

Priority	Config	Sig	S+V	N
Min size	$w=256, h=4$	2308 B	4.52 ms	16
Min compute	Lamp., $h=4$	24708 B	0.33 ms	16
Balanced	$w=16, h=5$	4452 B	0.61 ms	32
Max msgs	$w=16, h=8$	4548 B	0.61 ms	256

Table 4: Recommended configurations for different priorities (S+V = Sign + Verify time, N = max messages).

4 Mandatory Extension: Lamport Key-Reuse Attack

4.1 Threat Model and Theoretical Analysis

A Lamport secret key comprises $256 \times 2 = 512$ blocks. Each signature reveals exactly 256 blocks (one per row, selected by the corresponding message-hash bit). After k reuses with independent random messages, the probability that a *specific* block remains unrevealed is $(1/2)^k$. To forge a signature on an arbitrary target, the attacker needs all 256 blocks selected by the target's hash. The forgery probability is:

$$P_{\text{forge}}(k) = \left(1 - 2^{-k}\right)^{256} \tag{1}$$

We verified Eq. (1) empirically by running 200 independent trials per value of k, each with a fresh keypair (Table 5). As shown in Figure 8, the empirical forgery rate tracks the theoretical curve closely. After only ~10 reuses, the attacker succeeds with >78% probability; after 15, success is essentially certain. The attack is entirely *passive*—the attacker only needs to observe legitimate signatures without any interaction with the signer.

k	Theoretical	Empirical	Exposed
1	0.000	0.000	50.0%
4	0.000	0.000	93.9%
6	0.018	0.030	98.5%
8	0.367	0.385	99.6%
10	0.779	0.835	99.9%
12	0.939	0.950	100.0%
14	0.984	1.000	100.0%
20	1.000	1.000	100.0%

Table 5: Forgery probability vs. key reuses k. "Exposed" is the average fraction of all 512 secret blocks revealed to the attacker.

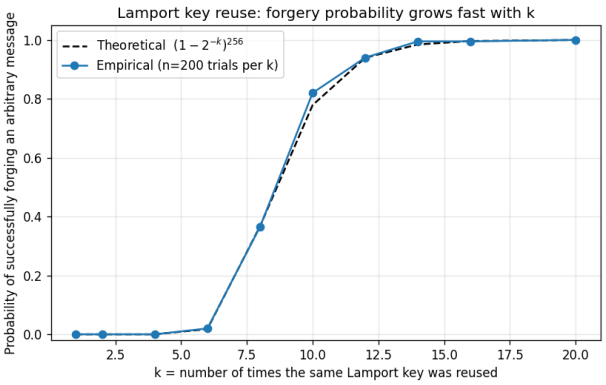

Figure 8: Forgery success rate vs. k. The empirical curve (200 trials per point) closely matches the theoretical prediction of Eq. (1), confirming the exponential convergence to certainty.

Figure 9 provides a complementary view: it plots the average fraction of the 512-block secret key exposed as a function of k. By $k = 8$, over 99.6% of blocks are revealed, and by $k = 12$ the entire key is typically exposed. This highlights that even a small number of reuses leaks nearly all secret material.

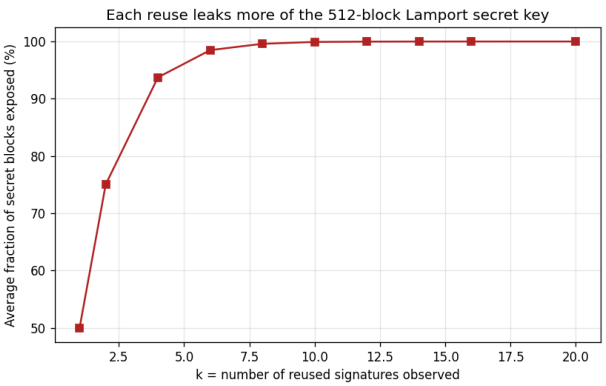

Figure 9: Fraction of Lamport secret-key blocks exposed vs. reuse count k. Convergence to 100% is rapid; by $k=12$ the attacker possesses the complete secret key.

4.2 Full Secret Key Recovery

We explicitly demonstrated secret-key recovery by signing 20 random messages with the same Lamport key. All 512 blocks were recovered byte-exact, as verified against the original key. The full recovered key was exported to JSON for inspection. Once in possession of the complete secret key, the attacker can forge a valid signature on *any* message.

5 Advanced Extension: Multi-Layer Merkle Trees (Hypertrees)

Section 3 (and Figure 3 in particular) exposed the dominant practical limitation of a flat MSS: KeyGen scales as $O(2^h)$, which becomes prohibitive once we want signing capacities of 2^{16} or beyond. We address this by implementing a two-layer *hypertree*, the same multi-layer Merkle construction used internally by SPHINCS+/SLH-DSA.

5.1 Construction

A two-layer hypertree consists of one *top tree* of height h_{top} whose leaves do not sign user messages directly; instead, each top-tree leaf certifies the public-key root of a *bottom tree* of height h_{bot}. Messages are signed by a leaf of the appropriate bottom tree, and the resulting signature is bundled with the top tree's MSS signature on that bottom tree's root. The total signing capacity is $N = 2^{h_{\text{top}}+h_{\text{bot}}}$, and the published public key is the top-tree root—a single 32-byte hash that commits to all N signing slots.

The decisive property is that bottom trees are built *lazily*: only the top tree and the first bottom tree are materialized at KeyGen time. This turns the up-front cost from $O(2^h)$ into roughly $O(2 \cdot 2^{h/2})$ when $h_{\text{top}} \approx h_{\text{bot}}$—a square-root reduction. Verification is unchanged in structure: the verifier checks both the bottom-tree MSS signature on the message and the top-tree MSS signature on the bottom-root, and ensures the bottom-tree index is consistent across the two.

5.2 Implementation and Tests

The construction lives in `src/hypertree.py` and is layered directly on top of our existing `MerkleSignature` class, requiring no modification to the underlying primitives. The `Hypertree` class exposes the standard `generate_keypair`, `sign`, and `verify` interface. Five unit tests in `tests/test_hypertree.py` confirm correctness, capacity exhaustion, and rejection of tampered messages and mismatched public keys; all five pass.

5.3 Empirical Results

Table 6 compares the two-layer hypertree against a flat MSS at matched signing capacities. WOTS ($w = 16$) leaves are used throughout. Figure 10 visualizes the KeyGen comparison.

Scheme	N	KG (ms)	Sign (ms)	Vrfy (ms)
Flat MSS, $h=8$	256	111.1	0.20	0.23
Hypertree, $h_t=h_b=4$	256	13.5	0.17	0.60
Flat MSS, $h=10$	1024	443.7	0.20	0.24
Hypertree, $h_t=h_b=5$	1024	28.5	0.19	0.43

Table 6: Hypertree vs flat MSS at matched signing capacities. KG = initial KeyGen time.

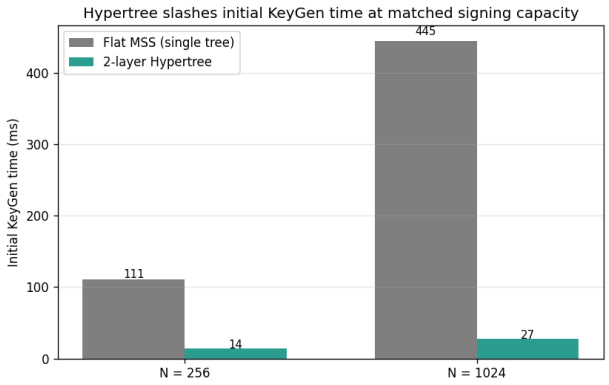

Figure 10: Initial KeyGen time: flat MSS vs 2-layer hypertree at matched signing capacities. The hypertree is 8.2× faster at $N=256$ and 15.6× faster at $N=1024$, confirming the predicted $O(\sqrt{N})$ behavior.

The data confirms three predictions. First, KeyGen time drops by roughly the expected $\sqrt{N}$ factor (8.2× at $N = 256$, 15.6× at $N = 1024$). Second, single-message Sign time is essentially unchanged, because signing still costs one bottom-tree MSS sign plus a precomputed top-tree certificate. Third, signature size roughly doubles (∼4.5 KB to ∼8.9 KB), since each hypertree signature now carries two MSS signatures instead of one—the standard size-vs-cost trade-off of multi-layer Merkle constructions. This is precisely the engineering trick that lets SPHINCS+ scale to $N=2^{64}$ while keeping initial KeyGen practical.

6 Conclusion

We implemented a complete hash-based signature pipeline from Lamport OTS through WOTS to the Merkle Signature Scheme, extended it to a two-layer hypertree, and supported all of it with comprehensive benchmarks and a quantitative key-reuse attack. Our key findings are:

- WOTS with $w = 16$ offers the best practical balance: 3.8× smaller signatures than Lamport with sign+verify under 0.6 ms.

- Flat MSS KeyGen scales as $O(2^h)$, while signing, verification, and signature size are nearly independent of tree height.
- Lamport key reuse is catastrophic: after $\sim$10 reuses, forgery succeeds with $>$78% probability; after 15 reuses, it is essentially certain. We recovered the full 512-block secret key byte-for-byte after only 20 reuses.
- The two-layer hypertree (Section 5) reduces initial KeyGen time by 8–16$\times$ at matched signing capacity, at the cost of roughly doubling the signature size—the same engineering trick used by SPHINCS+/SLH-DSA.

These results confirm that hash-based signatures are a viable and well-understood path to post-quantum security, requiring only the mild assumption of hash function one-wayness.

References

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proc. 35th Annual Symp. on Foundations of Computer Science (FOCS)*, pp. 124–134, IEEE, 1994.

[2] L. Lamport, "Constructing digital signatures from a one-way function," *SRI International, Technical Report CSL-98*, 1979.

[3] R. C. Merkle, "A certified digital signature," *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238, Springer, 1990.

[4] D. J. Bernstein et al., "SPHINCS+: Submission to the NIST post-quantum cryptography standardization project," 2017.